



ChatINDUS

TEXT TO LLM TO SQL

PROJET PIC

Rapport du Projet

CHATINDUS

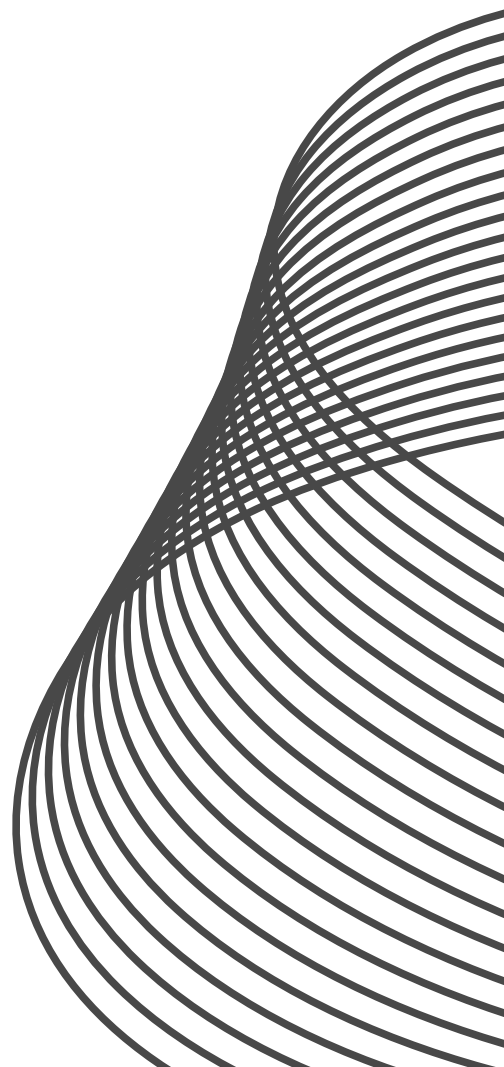
REALISER PAR :

- *EL HADDAD MARYAM*
- *EL MADRASSI ZAID*
- *REMIDI AYOUB*

ENCADRER PAR :

- *ESTEBAN BAUTISTA RUIZ*

2024-2025



Remerciements

Nous souhaitons tout d'abord exprimer notre profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réussite de ce projet.

Nous adressons nos remerciements les plus sincères à M. Esteban RUIZ BAUTISTA, notre tuteur pédagogique, pour son accompagnement, ses précieux conseils et son engagement tout au long de ce travail. Son expertise et sa disponibilité ont été essentielles à l'aboutissement de ce projet.

Nous tenons également à remercier l'ensemble des membres du laboratoire LISIC de l'EILCO pour leur aide, leur soutien et leur dévouement, qui nous ont permis de mener à bien cette étude dans un cadre propice à l'apprentissage et à la recherche.

Enfin, nous exprimons notre reconnaissance à tous nos professeurs de l'EILCO, dont les enseignements et les conseils avisés nous ont guidés tout au long de notre formation et nous ont permis d'acquérir les compétences nécessaires à la réalisation de ce projet.

Merci à toutes et à tous pour votre précieuse contribution.

Résumé Global du Projet

Dans de nombreux environnements industriels et professionnels, l'exploitation des données stockées dans des bases relationnelles repose sur la maîtrise du langage SQL. Cette compétence technique constitue un frein pour les utilisateurs non spécialistes qui souhaitent accéder rapidement à des informations pertinentes. Ainsi, le problème réel réside dans la nécessité de simplifier l'interrogation des bases de données sans exiger de compétences approfondies en SQL.

Plusieurs solutions existent actuellement pour répondre à cette problématique. Les assistants SQL alimentés par l'intelligence artificielle, tels que ChatGPT avec des plugins SQL, ou encore des plateformes de business intelligence (BI) comme Tableau ou Power BI, proposent des fonctionnalités d'interrogation simplifiée. D'autres outils comme Text-to-SQL Benchmarks utilisent des modèles de langage pour convertir des requêtes en langage naturel en requêtes SQL.

Cependant, ces solutions présentent des limites importantes. Elles sont souvent coûteuses, peu flexibles, ou nécessitent une configuration complexe. Certaines solutions n'offrent pas la possibilité d'exécution locale sécurisée, ce qui pose des problèmes de confidentialité des données. De plus, la vitesse de génération et d'exécution des requêtes peut être insuffisante pour des applications nécessitant une réponse en temps réel.

Pour répondre à ces insuffisances, ce rapport propose la solution ChatINDUS, qui se décline en deux versions complémentaires :

- Une solution locale avec PremSQL, offrant une exécution sécurisée et modulaire des requêtes SQL sur les infrastructures internes, idéale pour les environnements nécessitant une protection maximale des données.

- Une solution Cloud avec GroqCloud, exploitant des clés API pour garantir un accès rapide et sécurisé aux modèles IA avancés (Llama 3, Mixtral, Gemma) permettant la génération et l'optimisation des requêtes SQL en temps réel.

Un code launcher a été développé pour permettre à l'utilisateur de choisir facilement entre l'exécution locale et cloud. Ce launcher intègre également un système d'authentification sécurisée avec gestion des rôles et des accès aux bases de données, garantissant ainsi la protection et la confidentialité des informations traitées.

La méthodologie adoptée repose sur l'intégration de ces technologies dans une architecture modulaire :

- Interface avec Streamlit, divisée en modules pour une utilisation intuitive.
- Génération et exécution de requêtes SQL via GroqCloud et PremSQL, selon le mode choisi par l'utilisateur.
- Affichage des résultats sous forme de tableaux interactifs, facilitant l'analyse des données.
- Options de déploiement flexibles grâce au code launcher, garantissant des performances optimisées pour chaque environnement.

En conclusion, ChatINDUS constitue une solution complète et performante pour interagir avec des bases de données SQL en langage naturel. Elle se distingue par :

- Sa double approche locale et cloud, répondant aux différents besoins en matière de sécurité et de performance.
- Son interface intuitive, permettant une prise en main rapide.
- Sa rapidité d'exécution et sa flexibilité de déploiement, idéales pour les professionnels de la business intelligence et de l'analyse de données.

Sommaire

Remerciements.....	1
Résumé Global du Projet	1
Sommaire	3
Liste des figures	4
Liste des tableaux.....	5
I. Présentation du projet :.....	6
1. Présentation de l'équipe :.....	6
2. Contexte du projet :.....	6
3. Cahier de charges	7
4. Planning : Diagramme de Gantt :.....	8
5. Méthodologie :	9
6. QQQQCP :.....	10
II. Travail réalisé	11
1. Prérequis :	11
A. Visual Studio Code (VS Code) :	11
B. Python :.....	11
C. LLM :	12
D. PremSQL :	12
E. Groq et GroqCloud :.....	13
F. Streamlit :	14
2. Explication du Code Launcher.py :	14
A. Importation des Bibliothèques :	14
B. Affichage de l'image en haut de la page :	15
C. Sélection du Mode d'Exécution :	15
D. Bouton de Lancement de l'Application :	16
E. Footer Fixe en Bas de Page :	16
F. Conclusion :	17
3. Explication du Code Cloud.py :	18
A. Importation des Bibliothèques :	18
B. Affichage de l'image en haut de la page :	18
C. Authentification des utilisateurs :	18
D. Affichage de la Barre Latérale (Sidebar) :	19
E. Définition de la Clé API pour ChatGroq :	20
F. Connexion et Chargement de la Base de Données :	20
G. Génération de Requêtes SQL avec l'IA :	21
H. Chat & Exécution de la Requête :	22
I. Footer & Ajout d'une Image dans la Sidebar :	22
J. Conclusion :	22

4. Explication du Code Local.py	23
A. Importation des Bibliothèques :	23
B. Affichage de l'image en haut de la page :	23
C. Authentification des utilisateurs :	23
D. Affichage de la Barre Latérale (Sidebar) :	23
E. Initialisation des Générateurs Text-to-SQL :	23
F. Sélection de la Base de Données :	24
G. Initialisation du Worker (Traitement Text-to-SQL) :	24
H. Gestion des Messages et Historique :	24
I. Affichage des Messages Précédents :	25
J. Traitement d'une Nouvelle Question :	25
K. Génération et Exécution de la Requête SQL :	26
L. Vérification et Exécution de la Requête SQL :	26
M. Footer Fixe et Image dans la Sidebar :	27
N. Conclusion :	27
5. Présentation de l'Interface Streamlit des Applications ChatINDUS :	28
A. Structure Générale de l'Interface :	28
B. Barre Latérale (Sidebar) :	28
C. Zone Centrale (Corps de l'Application) :	28
D. Expérience Utilisateur et Ergonomie :	28
E. Élément Visuel Important : Le Footer :	29
F. Conclusion :	29
6. Axes d'Amélioration pour les Applications ChatINDUS :	29
A. Amélioration de l'Expérience Utilisateur :	29
B. Optimisation des Performances :	30
C. Sécurité et Gestion des Accès :	30
D. Déploiement et Accessibilité :	31
7. Conclusion	31
III. Conclusions et perspectives :	31
IV. Annexes	32

Liste des figures

Figure 1: Présentation des membres de l'équipe	6
Figure 2 : Diagramme de Gantt prévisionnel	8
Figure 3 : Diagramme de Gantt prévu	9
Figure 4 : Importation des Bibliothèques	14
Figure 5 : Affichage de l'image en haut de la page	15
Figure 6 : Sélection du Mode d'Exécution	15
Figure 7 : Bouton de Lancement de l'Application	16
Figure 8 : Footer Fixe en Bas de Page	16
Figure 9 : Interface launcher	17
Figure 10 : Importation des Bibliothèques	18

Figure 11 : User roles	18
Figure 12 : Authentification des utilisateurs	19
Figure 13 : Affichage le titre de l'application (ChatINDUS)	19
Figure 14 : Bouton externe pour visualiser la BD	19
Figure 15 : NLP to SQL	20
Figure 16 : Champ pour la Clé API	20
Figure 17 : Champs pour insérer le model AI	20
Figure 18 : Connexion avec la BD	20
Figure 19 : Charge le schéma de la BD	21
Figure 20 : Model de requête SQL.....	21
Figure 21 : Génération d'une réponse en NLP	21
Figure 22 : Commande pour l'historique.....	22
Figure 23 : Commande pour écrire en NLP	22
Figure 24 : Convertit la question en SQL	22
Figure 25 : Importation des Bibliothèques	23
Figure 26 : Initialisation des Générateurs Text-to-SQL	23
Figure 27 : Sélection de la Base de Données.....	24
Figure 28 : Traitement Text-to-SQL.....	24
Figure 29 : Gestion des Messages et Historique	24
Figure 30 : Affiche les messages	25
Figure 31 : Affiche les résultats SQL	25
Figure 32 : Champ de saisie en NLP.....	25
Figure 33 : Stocke la requête dans l'historique	25
Figure 34 : Génération et Exécution de la Requête SQL	26
Figure 35 : Vérification	26
Figure 36 : Affiche la Requête SQL	26
Figure 37 : Exécute la requête SQL.....	27
Figure 38 : Ajoute la réponse du bot.....	27
Figure 39 : Schéma générale du projet.	32
Figure 40 : Annexe 1	32

Liste des tableaux

Tableau 1 : QQQQCP	10
--------------------------	----

I. Présentation du projet :

1. Présentation de l'équipe :

Notre équipe est composée de trois étudiants en troisième année de génie industriel à l'EILCO, partageant la même spécialisation en écologie industrielle, ce qui nous a naturellement réunis pour ce projet.

Nous avons eu l'honneur de travailler sous l'encadrement de M. Esteban RUIZ BAUTISTA, maître de conférences à l'ULCO, au sein du laboratoire LISIC, dont l'accompagnement a été précieux pour la réussite de ce projet.

Vous trouverez ci-dessous la liste des membres de notre équipe :

NOTRE ÉQUIPE



• **Maryam EL HADDAD**



• **Zaid EL MADRASSI**



• **Ayoub REMIDI**

Figure 1: Présentation des membres de l'équipe

2. Contexte du projet :

Nous évoluons dans un contexte où une quantité toujours croissante d'informations est générée, en particulier par les entreprises. Ces données proviennent de diverses sources, telles que la production, la logistique, la relation client ou encore la maintenance. Exploiter ces données de manière efficace est devenu un enjeu majeur pour les organisations souhaitant améliorer leur compétitivité.

Aujourd'hui, les entreprises tirent de la valeur de ces données en les analysant afin de prendre de meilleures décisions stratégiques et opérationnelles. Cette analyse permet d'optimiser les processus, de réduire les coûts, d'améliorer la qualité des produits et services, et d'anticiper les besoins du marché.

Ces informations sont généralement stockées dans des bases de données relationnelles. Cependant, ces bases restent très obscures pour les employés non techniques, car leur interrogation nécessite la maîtrise du langage SQL. Cette compétence technique représente un obstacle important, ralentissant l'accès aux données et freinant la prise de décision rapide.

Une solution existante consiste à développer une interface logicielle permettant aux employés d'accéder aux données via des menus et des champs textuels. En arrière-plan,

l'interface traduit ces choix en requêtes SQL exécutées sur la base de données. Toutefois, cette approche présente des limites :

- Les entreprises restent dépendantes du développeur ayant conçu l'interface, ce qui complique les évolutions futures.
- L'interface peut ne pas permettre de formuler des requêtes complexes, limitant ainsi la profondeur de l'analyse des données.

Face à ces limitations, nous proposons ChatINDUS, un traducteur capable de transformer directement une question posée en langage naturel par un employé en une requête SQL appropriée. Cette solution vise à démocratiser l'accès aux données pour tous les collaborateurs, qu'ils soient techniques ou non.

Pour cela, nous exploitons la puissance des LLMs (Large Language Models), qui ont récemment démontré des performances impressionnantes en compréhension et génération de texte. Ces modèles permettent de générer des requêtes SQL optimisées et pertinentes en temps réel, sans nécessiter d'interventions techniques spécifiques.

Nous espérons que cet outil révolutionnera l'accès aux données en entreprise, en facilitant les prises de décision rapides et en boostant significativement la productivité. En supprimant les barrières techniques à l'analyse des données, ChatINDUS s'inscrit comme une solution innovante pour accompagner les entreprises dans leur transformation numérique.

3. Cahier de charges

L'objectif principal de ce projet est de démontrer que les Large Language Models (LLMs) peuvent constituer d'excellents traducteurs de texte en langage naturel vers des requêtes SQL exploitables. Pour atteindre cet objectif, le projet a été structuré en plusieurs étapes principales.

- **Apprentissage et maîtrise de Python :**

La première phase a consisté à se familiariser avec Python, son environnement de développement (VS Code) et les bibliothèques nécessaires telles que Torch Transformers et Premsql, afin d'assurer une intégration fluide des technologies.

- **Compréhension du fonctionnement des LLMs :**

Une étude approfondie du fonctionnement des LLMs a été réalisée, notamment grâce à la lecture du livre *Build LLMs from Scratch*. Cette lecture a permis d'acquérir une compréhension des concepts clés tels que l'architecture des transformers, les mécanismes d'attention et les étapes d'entraînement, influençant directement nos choix méthodologiques et technologiques.

- **Conception du pipeline logiciel :**

Le pipeline logiciel a été conçu pour répondre aux besoins identifiés. Deux solutions complémentaires ont été développées : une solution locale avec PremSQL, garantissant une exécution sécurisée sur une infrastructure interne, et une solution cloud avec GroqCloud, exploitant des clés API pour un accès flexible. Ces solutions sont reliées par un code launcher

permettant à l'utilisateur de sélectionner le mode d'exécution souhaité, tout en assurant une authentification sécurisée.

- **Développement de l'interface utilisateur :**

Une interface ergonomique a été développée avec Streamlit pour garantir une interaction intuitive avec l'outil. Cette interface offre une visualisation claire des résultats sous forme de tableaux interactifs, facilitant l'analyse des données.

- **Optimisation des performances :**

Des optimisations ont été mises en place afin d'assurer une exécution rapide des requêtes, une gestion efficace des ressources et une sécurité renforcée dans les deux environnements d'exécution.

- **Livrables :**

Les livrables du projet incluent un poster de présentation synthétisant les objectifs, la méthodologie et les résultats clés, ainsi qu'un rapport détaillé couvrant l'ensemble du processus de développement. La livraison finale est prévue pour le 23 février 2024, conformément aux délais impartis.

Ce cahier des charges présente donc les étapes essentielles permettant de démontrer que les LLMs peuvent révolutionner l'accès aux données en entreprise en offrant une interaction naturelle et intuitive avec les bases SQL.

4. Planning : Diagramme de Gantt :

Dès le début de notre projet, nous avons établi un diagramme de Gantt prévisionnel afin de mieux nous organiser dans la réalisation des différentes étapes et d'attribuer le temps nécessaire à chaque tâche.

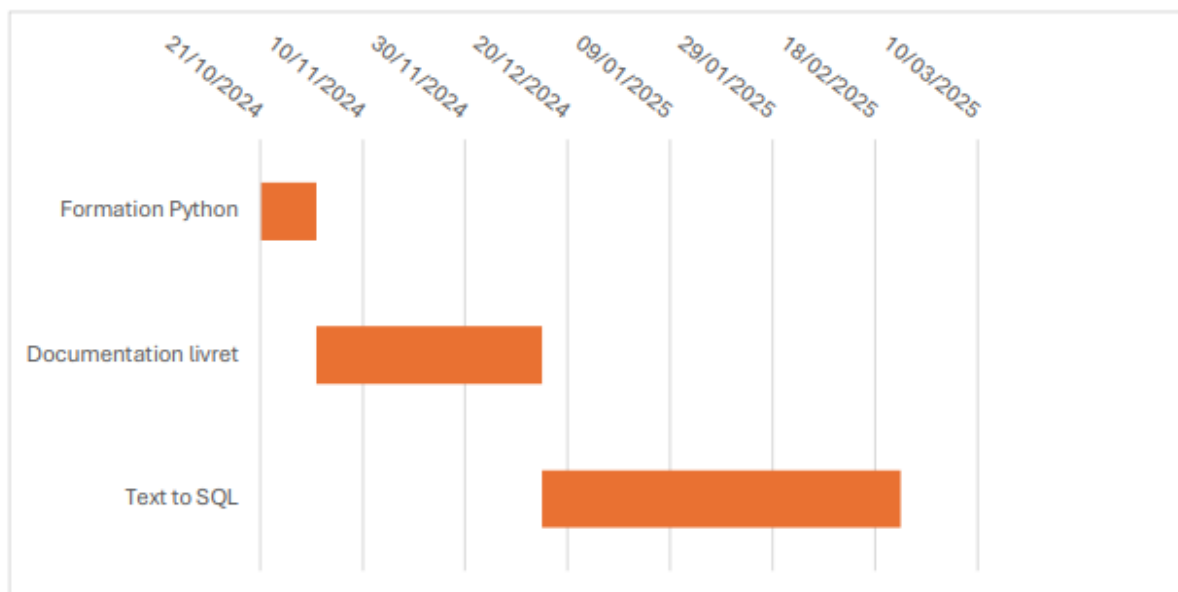


Figure 2 : Diagramme de Gantt prévisionnel

Au cours de la réalisation du projet, certaines tâches ont pris plus de temps que prévu, tandis que d'autres se sont avérées plus rapides. De plus, nous avons découvert l'existence de tâches intermédiaires qui n'avaient pas été anticipées. Finalement, d'après le diagramme de Gantt ci-dessous, nous avons ajusté notre planification en conséquence.

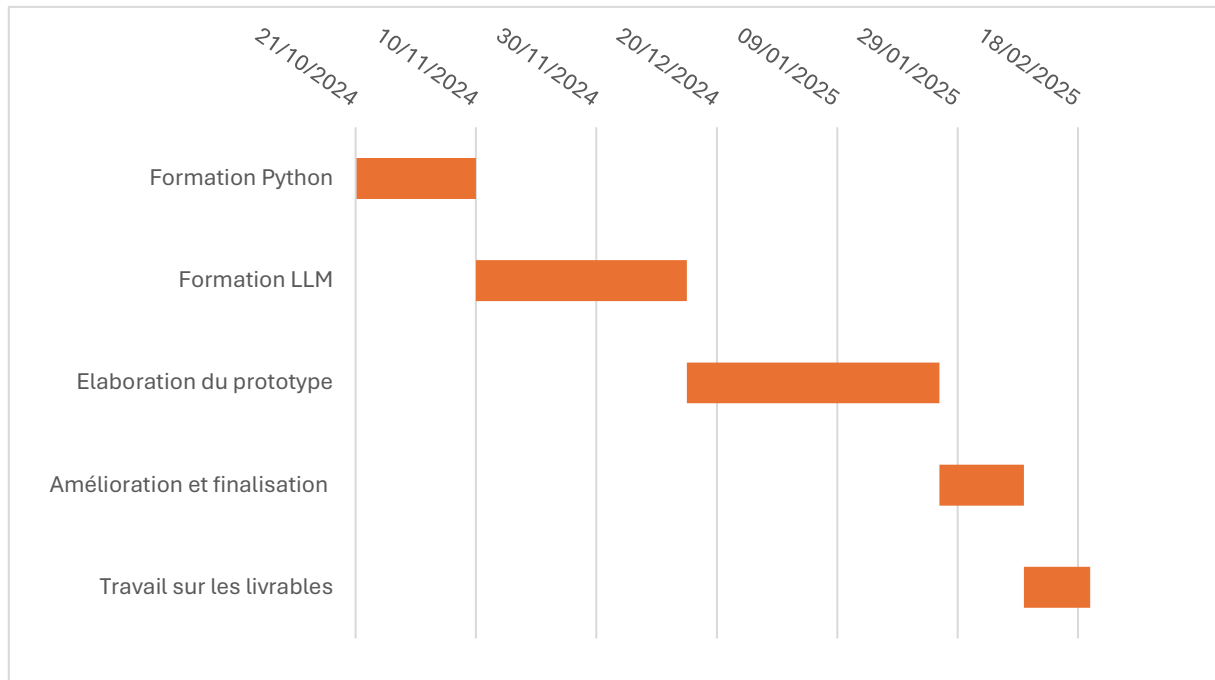


Figure 3 : Diagramme de Gantt prévu

5. Méthodologie :

Cette section détaille la méthodologie adoptée pour la mise en œuvre du projet ChatINDUS, visant à transformer des requêtes en langage naturel en requêtes SQL exploitables grâce à l'utilisation de Large Language Models (LLMs).

Nous avons choisi d'utiliser un LLM pré-entraîné pour la tâche Text2SQL, en l'occurrence PremSQL. Ce modèle a été sélectionné pour ses performances en génération de requêtes SQL à partir d'instructions en langage naturel, tout en garantissant une exécution locale sécurisée.

Pour intégrer ce modèle, nous avons développé une interface utilisateur avec Streamlit, permettant à l'utilisateur de soumettre directement ses requêtes en langage naturel. Cette interface assure une interaction fluide et intuitive avec l'outil. Une fois la requête soumise, elle est traitée par des workers PremSQL, qui jouent un rôle clé en assurant la communication entre le modèle LLM et la base de données SQL. Cette étape garantit également l'analyse et la présentation des résultats obtenus.

Au cours du développement, nous avons constaté que cette solution fonctionnait exclusivement en environnement local, ce qui limitait la flexibilité du déploiement. Pour surmonter cette limitation, nous avons intégré GroqCloud, une solution permettant une exécution dans le cloud. Bien que cette approche implique l'utilisation d'un modèle moins

spécialisé, elle offre une plus grande flexibilité d'accès et d'exécution, répondant ainsi à des besoins variés en entreprise.

Le processus global a été synthétisé dans le pipeline présenté en Figure 39 (ci-dessous), illustrant la séquence complète, de la soumission de la requête par l'utilisateur à la génération de la réponse finale. Ce pipeline met en évidence :

- L'utilisation d'un code launcher permettant de sélectionner le mode d'exécution (local ou cloud).
- L'étape d'authentification sécurisée, garantissant la gestion des accès aux bases de données.
- La conversion des questions en requêtes SQL par le modèle LLM.
- L'extraction des données pertinentes et l'affichage des résultats sous forme de tableaux interactifs.

Cette méthodologie offre ainsi une solution robuste et flexible, combinant les avantages d'une exécution locale sécurisée avec PremSQL et la souplesse du déploiement cloud via GroqCloud.

6. QQQQCP :

Pour mieux nous positionner dans le projet et en avoir une vision prospective, nous vous proposons ce QQQQCCP du projet ChatINDUS.

Voici le QQQQCP de notre projet :

Qui	Maryam EL HADDAD, Zaid EL MADRASSI, et Ayoub REMIDI.
Quoi	Développer un outil permettant d'interroger une base de données en langage naturel et d'obtenir des réponses précises et immédiates.
Où	Laboratoire LISIC de l'école.
Quand	Du 21 octobre au 23 février.
Comment	À l'aide du LLM et de modèles pré-entraînés, permettre de formuler des questions en langage naturel et d'obtenir des réponses précises.
Pourquoi	Faciliter l'accès aux bases de données , améliorer l'efficacité opérationnelle et soutenir la prise de décision.

Tableau 1 : QQQQCP

II. Travail réalisé

1. Prérequis :

A. Visual Studio Code (VS Code) :

Dans notre projet, nous avons utilisé Visual Studio Code (VS Code) comme environnement de développement principal. Cet éditeur de code, développé par Microsoft, est largement adopté pour le développement en Python grâce à ses nombreuses fonctionnalités avancées. Il offre une interface fluide et extensible qui permet d'améliorer la productivité grâce à une large gamme d'extensions adaptées aux besoins des développeurs.

Son support natif de Python, notamment via l'extension officielle dédiée, nous a facilité l'écriture, l'exécution et le débogage de nos scripts. De plus, l'intégration d'un terminal directement dans l'éditeur nous a permis d'exécuter nos scripts Python sans avoir à passer par un terminal externe, tout en interagissant facilement avec les bases de données utilisées dans notre projet. Par ailleurs, l'intégration native avec Git nous a permis de gérer efficacement le versioning de notre code, en suivant les évolutions et en facilitant la collaboration entre les membres de l'équipe. L'utilisation de VS Code nous a offert un environnement de travail unifié et efficace, nous permettant d'assurer une meilleure organisation et fluidité dans la modélisation, le développement et les tests de notre application.

B. Python :

Python est un langage de programmation interprété, polyvalent et largement utilisé dans divers domaines de l'informatique. Il se distingue par sa syntaxe simple et intuitive, favorisant ainsi une prise en main rapide et une meilleure lisibilité du code, ce qui le rend accessible aussi bien aux débutants qu'aux développeurs expérimentés. Grâce à sa flexibilité et à son vaste écosystème de bibliothèques et frameworks, Python est particulièrement adapté au développement web, à l'analyse de données, à l'intelligence artificielle, à l'automatisation des tâches et à l'administration des systèmes. Son interopérabilité avec d'autres langages et technologies en fait un choix privilégié pour des projets nécessitant des intégrations variées.

Dans notre projet, Python a joué un rôle central en raison de sa simplicité et de sa capacité à manipuler efficacement des bases de données SQL. Son large éventail de bibliothèques nous a permis d'intégrer des outils avancés facilitant le traitement du langage naturel et l'interaction avec notre base de données. Nous avons notamment utilisé la bibliothèque `sqlite3` pour assurer une gestion fluide des requêtes SQL et automatiser l'exécution des requêtes générées. Grâce à Python, nous avons développé un outil performant capable de convertir des requêtes en langage naturel en requêtes SQL optimisées, garantissant ainsi une meilleure accessibilité aux bases de données. Son intégration fluide avec Visual Studio Code (VS Code) nous a permis d'écrire, tester et améliorer notre application de manière efficace, assurant un développement structuré et optimisé tout au long du projet.

C. LLM :

Le livre, "Build a Large Language Model (From Scratch)" de Sebastian Raschka, nous a été donné par Monsieur Esteban afin de nous aider à mieux comprendre les Large Language Models (LLM) et leur fonctionnement en profondeur. Grâce à cet ouvrage, nous avons pu explorer les concepts fondamentaux, les techniques avancées et les différentes étapes nécessaires à la construction d'un modèle de langage à partir de zéro.

Le livre est une ressource précieuse qui nous a permis d'approfondir notre compréhension des LLMs, en particulier en ce qui concerne la préparation des données textuelles, l'implémentation des mécanismes d'attention et la conception d'un modèle de type GPT. Nous avons focalisé notre lecture sur l'Appendix A ainsi que sur les chapitres 1, 2, 3 et 4, qui constituent la base essentielle pour comprendre la structure et l'entraînement d'un LLM.

Notre lecture s'est ainsi articulée autour des points suivants :

- Chapitre 1 : Comprendre les grands modèles de langage. Ce chapitre définit ce qu'est un LLM, explique son architecture de base (transformer), ses applications et décrit les différentes étapes de construction d'un modèle.
- Chapitre 2 : Travailler avec des données textuelles. Il couvre la préparation des données textuelles, notamment la tokenisation, l'encodage en embeddings, et l'échantillonnage des données pour alimenter un modèle.
- Chapitre 3 : Implémentation des mécanismes d'attention. Il explore le fonctionnement de l'attention dans les modèles de langage et détaille l'implémentation des self-attention layers, y compris l'attention multi-tête.
- Chapitre 4 : Implémentation d'un modèle GPT à partir de zéro. Ce chapitre se concentre sur l'implémentation complète d'un modèle de type GPT, en détaillant les blocs essentiels d'un transformer, l'utilisation des couches d'attention et de normalisation, ainsi que le processus de génération de texte.

Grâce à cette lecture, nous avons pu acquérir une compréhension approfondie des principes et des techniques utilisés pour concevoir et entraîner un LLM, ce qui nous a aidés dans notre travail sur ce sujet.

D. PremSQL :

PremSQL est une bibliothèque open-source qui permet de convertir des requêtes en langage naturel en requêtes SQL exécutables, garantissant une analyse de données sécurisée et autonome. Son architecture modulaire repose sur des modèles de génération Text-to-SQL, prenant en charge PremAI, HuggingFace, OpenAI, et permet une connexion avec SQLite, PostgreSQL, MySQL, ainsi que l'importation de fichiers CSV. Elle inclut des fonctionnalités avancées comme la validation et correction automatique des requêtes SQL, la génération de rapports analytiques et l'intégration de visualisations avec Matplotlib.

PremSQL offre une interface interactive qui permet aux utilisateurs d'exécuter des requêtes SQL via un Playground, similaire à ChatGPT, et prend en charge la gestion des jeux

de données en facilitant l'importation et la création de datasets pour l'entraînement des modèles SQL. Son installation est simple et rapide via pip, nécessitant Python 3.8+.

Les cas d'utilisation de PremSQL couvrent divers domaines comme l'analyse des ventes (suivi des performances commerciales), l'automatisation des rapports financiers (analyse des dépenses et profits), et l'assistance aux développeurs (génération et correction automatique des requêtes SQL). Ses principaux avantages résident dans sa confidentialité, sa facilité d'utilisation et son système de correction intelligent, bien qu'il dépende des modèles LLM pour la précision et nécessite une configuration initiale.

En conclusion, PremSQL est une solution flexible et efficace qui permet aux entreprises de générer, exécuter et analyser des requêtes SQL de manière sécurisée et optimisée, sans compromettre la confidentialité des données.

E. [Groq et GroqCloud](#) :

Fondée en 2016 par d'anciens ingénieurs de Google, Groq est une entreprise spécialisée dans le développement de matériels et logiciels d'intelligence artificielle (IA) à haute performance. Son infrastructure est conçue pour optimiser l'inférence IA, notamment dans le traitement et l'optimisation des requêtes SQL. GroqCloud, sa plateforme cloud, permet aux développeurs d'exploiter la puissance des LPUs (Language Processing Units) pour générer, traduire et exécuter des requêtes SQL en temps réel. En adoptant une approche software-first, GroqCloud assure une parfaite synergie entre le développement logiciel et l'optimisation matérielle, garantissant une exécution rapide et une gestion efficace des ressources énergétiques.

GroqCloud prend en charge plusieurs modèles de langage avancés, tels que Gemma 2 (Google), Llama 3.3 (Meta) et Mixtral 8x7B (Mistral), chacun étant spécialisé dans des tâches précises allant de la génération SQL avancée à l'exécution de requêtes complexes. Ces modèles sont particulièrement adaptés aux domaines de la Business Intelligence, de l'automatisation SQL et de l'optimisation des bases de données web, avec une compatibilité étendue aux architectures SQL telles que SQL Server, PostgreSQL et MySQL. Toutefois, l'utilisation de modèles IA pour la génération des requêtes nécessite une supervision humaine, et l'adoption de GroqCloud demande une adaptation progressive aux environnements de bases de données complexes.

Pour exploiter GroqCloud, il est nécessaire d'obtenir [une clé API](#), accessible depuis la console développeur. Une fois connecté, l'utilisateur peut générer une clé [API](#) sécurisée et consulter la page des modèles disponibles afin de sélectionner celui qui correspond le mieux à ses besoins. Chaque modèle est identifié par un ID unique, permettant une intégration fluide avec [l'API](#). Par exemple, pour utiliser le modèle "llama-3.3-70b-versatile", il suffit de spécifier son ID dans les requêtes envoyées à [l'API](#).

En matière de performance et benchmarking, GroqCloud est comparé à d'autres solutions Text-to-SQL telles que LangChain SQL Agent, ChatDB et SQLCoder, où il se distingue par une rapidité accrue grâce à l'utilisation des LPUs et une meilleure optimisation via LangChain. En conclusion, GroqCloud se positionne comme une solution de pointe pour

automatiser et optimiser le traitement des requêtes SQL, offrant aux développeurs et analystes une plateforme puissante, flexible et hautement performante pour l'analyse et l'exploitation des bases de données.

F. [Streamlit](#) :

Streamlit est un framework open-source en Python qui permet de créer des applications web interactives avec un minimum de code. Destiné aux data scientists, analystes de données et ingénieurs en IA, il simplifie la visualisation et l'analyse des données sans nécessiter de compétences avancées en développement web. Son principal atout est sa rapidité de développement, permettant de transformer un simple script Python en une application fonctionnelle en quelques minutes.

Grâce à une installation simple et une compatibilité native avec des bibliothèques comme Pandas, NumPy, TensorFlow et Matplotlib, Streamlit offre une interface fluide avec des widgets interactifs (boutons, sliders, formulaires) et un rafraîchissement automatique des modifications. Son architecture légère facilite le déploiement sur diverses plateformes telles que Streamlit Cloud, Heroku et Docker.

Ses applications couvrent plusieurs cas d'usage : analyse de données avancée, prototypage de modèles d'IA, visualisation en temps réel et création d'outils internes pour les entreprises. Pour débiter, une simple installation via `pip install streamlit` suffit, suivie de la création d'un fichier `app.py` contenant le script de l'application. Une fois exécutée, l'application s'ouvre directement dans un navigateur web, rendant son utilisation intuitive.

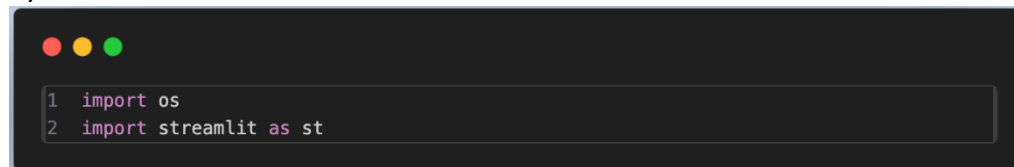
Streamlit est une solution idéale pour accélérer le développement d'applications interactives, améliorer la présentation des résultats d'analyse et démocratiser l'utilisation des données grâce à une interface accessible et intuitive.

2. [Explication du Code Launcher.py](#) :

Ce script utilise Streamlit pour créer une interface web interactive permettant aux utilisateurs de choisir entre deux modes d'exécution (Local et Cloud) et de lancer l'application en conséquence. Il intègre également une image en haut de la page et un footer fixe en bas.

A. [Importation des Bibliothèques](#) :

Python



```
1 import os
2 import streamlit as st
```

Figure 4 : Importation des Bibliothèques

- `Os` : Fournit des fonctionnalités pour interagir avec le système d'exploitation (exécuter des commandes, vérifier l'existence de fichiers, etc.).
- `Streamlit as st` : Importation du framework Streamlit pour créer l'interface web interactive.

B. Affichage de l'image en haut de la page :

Python

```
1 with st.container():
2     image_path = "labo.jpeg"
3     if os.path.exists(image_path):
4         st.image(image_path, use_container_width=True)
5     else:
6         st.warning(f"L'image {image_path} est introuvable.")
```

Figure 5 : Affichage de l'image en haut de la page

Explication :

- `st.container()`: Permet de structurer le contenu dans un conteneur.
- Vérification si l'image labo.jpeg existe dans le répertoire :
 - Si oui → Affiche l'image avec `st.image(image_path, use_container_width=True)`.
 - Si non → Affiche un message d'avertissement `st.warning(...)`.

Objectif : Afficher une bannière ou un logo en haut de la page.

C. Sélection du Mode d'Exécution :

Python

```
1 st.title("🖥️ Sélection du mode d'exécution ")
2 type_execution = st.radio("Choisissez le mode d'exécution", ["Local", "Cloud"])
```

Figure 6 : Sélection du Mode d'Exécution

Explication :

- `st.title(...)` : Affiche un titre en haut de la page.
- `st.radio(...)` : Crée un bouton radio permettant à l'utilisateur de choisir entre :
 - Local
 - Cloud

Objectif : Permettre à l'utilisateur de sélectionner le mode dans lequel il souhaite exécuter l'application.

D. Bouton de Lancement de l'Application :

Python

```
1 if st.button("Lancer l'application"):
2     if type_execution == "Local":
3         st.write("♦ Lancement de la version local    ")
4         os.system("streamlit run local.py") # Exécute le fichier local.py
5     elif type_execution == "Cloud":
6         st.write("♦ Lancement de la version clou    ")
7         os.system("streamlit run cloud.py") # Exécute le fichier cloud.py
8     else:
9         st.error("❌ Choix invalide. Relancez et sélectionnez Local ou Clou
                                d.")
```

Figure 7 : Bouton de Lancement de l'Application

Explication :

- st.button("Lancer l'application") : Affiche un bouton sur lequel l'utilisateur peut cliquer pour exécuter l'application.
- Lorsque l'utilisateur clique :
- Si le choix est "Local" → Affiche " ♦ Lancement de la version locale..." et exécute local.py en arrière-plan.
- Si le choix est "Cloud" → Affiche " ♦ Lancement de la version cloud..." et exécute cloud.py.
- Sinon → Affiche un message d'erreur si aucun choix valide n'est sélectionné.

Objectif : Lancer l'application en local ou dans le cloud selon la sélection de l'utilisateur.

E. Footer Fixe en Bas de Page :

Python

```
1 with st.container():
2     st.markdown("""
3         <style>
4             .footer {
5                 position: fixed;
6                 bottom: 0;
7                 width: 100%;
8                 background-color: white;
9                 text-align: center;
10                padding: 10px;
11                font-size: 12px;
12                border-top: 1px solid #ddd;
13                z-index: 100;
14            }
15        </style>
16        <div class='footer'>
17            © MADE BY : EL HADDAD / EL MADRASSI / REMIDI<br>
18            © SUPERVISED BY : Mr.ESTEBAN RUIZ BAUTISTA
19        </div>
20    , unsafe_allow_html=True)
```

Figure 8 : Footer Fixe en Bas de Page

Explication :

- `st.container()`: Crée un conteneur pour structurer le footer.
- `st.markdown(...)` :
 - Utilisation du HTML + CSS pour fixer un footer en bas de la page.
 - Ajoute un texte avec les auteurs du projet et le superviseur.
 - `unsafe_allow_html=True` permet d'exécuter du code HTML/CSS dans Streamlit.

Objectif : Ajouter un footer personnalisé et fixe en bas de la page pour afficher les crédits du projet.

F. Conclusion :

Ce code crée une interface interactive avec Streamlit qui permet :

- D'afficher une image fixe en haut.
- De choisir un mode d'exécution (Local/Cloud) via des boutons radio.
- De lancer l'application en fonction du choix de l'utilisateur.
- D'ajouter un footer fixe avec des informations sur les auteurs du projet.



Figure 9 : Interface launcher

3. Explication du Code Cloud.py :

A. Importation des Bibliothèques :

Python

```
1 import os
2 import streamlit as st
3 import pandas as pd
4 import sqlite3
5 from langchain_groq import ChatGroq
6 from langchain.sql_database import SQLDatabase
7 from langchain.chains import LLMChain
8 from langchain.prompts import PromptTemplate
```

Figure 10 : Importation des Bibliothèques

Explication :

- Os : Pour gérer les interactions avec le système (exécution de commandes, variables d'environnement).
- Streamlit as st : Framework pour créer l'interface utilisateur.
- Pandas as pd : Pour manipuler les données sous forme de DataFrame.
- Sqlite3 : Pour interagir avec les bases de données SQLite.
- ChatGroq : Utilise Llama-3.3-70B pour interagir avec une IA en langage naturel.
- SQLDatabase : Permet d'interagir avec une base de données SQL.
- LLMChain : Enchaîne les interactions entre l'IA et le SQL.
- PromptTemplate : Définit des modèles de requêtes pour générer du SQL et des réponses en langage naturel.

B. Affichage de l'image en haut de la page :

Explication : Cette partie a déjà été expliquée dans le launcher.

C. Authentification des utilisateurs :

Python

```
1 if "authenticated" not in st.session_state:
2     st.session_state.authenticated = False
3     st.session_state.user_role = None
4     st.session_state.allowed_databases = []
5
6 user_roles = {
7     "esteban": {"password": "esteban", "databases": ["car_retails.sqlite",
8     "human_resources.sqlite", "sales.sqlite"]},
9     "admin": {"password": "password", "databases": ["car_retails.sqlite",
10    "human_resources.sqlite", "sales.sqlite"]},
11    "rh": {"password": "rhpass", "databases": ["human_resources.sqlite",
12    "sales.sqlite"]},
13    "finance": {"password": "finpass", "databases": ["sales.sqlite"]}
14 }
```

Figure 11 : User roles

Explication :

- Gestion de session : st.session_state stocke l'état de connexion et les permissions de l'utilisateur.
- Dictionnaire user_roles : Définit les rôles des utilisateurs avec leurs mots de passe et les bases de données qu'ils peuvent accéder.

Python

```
1 if not st.session_state.authenticated:
2     st.title("🔒 Authentification")
3     username = st.text_input("Nom d'utilisateur", "")
4     password = st.text_input("Mot de passe", "", type="password")
5     if st.button("Se connecter"):
6         if username in user_roles and password == user_roles[username]["password"]
7     ]:
8         st.session_state.authenticated = True
9         st.session_state.user_role = username
10        st.session_state.allowed_databases = user_roles[username]["databases"]
11    ]
12    st.rerun()
13 else:
14     st.error("Nom d'utilisateur ou mot de passe incorrect")
15 st.stop()
```

Figure 12 : Authentification des utilisateurs

- Affiche un formulaire de connexion (st.text_input pour le nom et le mot de passe).
- Vérifie l'identité de l'utilisateur et les bases de données accessibles.

D. Affichage de la Barre Latérale (Sidebar) :

Python

```
1 st.sidebar.markdown("""
2     <div style='text-align: center;'>
3     <h1>🗨️ ChatINDUS - CLOUD</h1>
4     > </div>
5     """, unsafe_allow_html=True)
```

Figure 13 : Affichage le titre de l'application (ChatINDUS)

Affiche le titre de l'application dans la barre latérale.

python

```
1 st.sidebar.markdown("""
2     <a href='https://filext.com/fr/online-file-viewer.html' target='_blank'>
3     <button style='width: 100%; padding: 10px; font-size: 16px; background-color: #007bff; color: white; border: none; cursor: pointer;'>
4     📄 Visualiser la Base de donnée
5     </button>
6     </a>
7     """, unsafe_allow_html=True)
```

Figure 14 : Bouton externe pour visualiser la BD

Ajoute un bouton cliquable permettant d'accéder à un site externe pour visualiser la base de données.

Python

```
1 st.sidebar.write(
    "Pose une question en langage naturel et obtiens une requête SQL exécutée sur la b
    ase de données."
)
```

Figure 15 : NLP to SQL

Affiche une description expliquant que l'application convertit le langage naturel en requêtes SQL.

E. Définition de la Clé API pour ChatGroq :

Python

```
1 os.environ["GROQ_API_KEY"] =
    "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"
```

Figure 16 : Champ pour la Clé API

Stocke [la clé API](#) dans les variables d'environnement pour permettre l'accès à l'IA ChatGroq.

Python

```
1 llm = ChatGroq(model="llama-3.3-70b-versatile")
```

Figure 17 : Champs pour insérer le model AI

Initialise un modèle IA basé sur [Llama 3.3-70B](#).

F. Connexion et Chargement de la Base de Données :

Python

```
1 selected_db = st.sidebar.selectbox("Sélectionnez une base de données"
    , st.session_state.allowed_databases)
2 DB_PATH = selected_d
3 conn = sqlite3.connect(DB_PATH)
```

Figure 18 : Connexion avec la BD

Sélectionne une base de données parmi celles autorisées à l'utilisateur.
Établit une connexion avec SQLite.

Python

```
1 db = SQLiteDatabase.from_uri(f"sqlite://{DB_PATH}")
2 database_schema = db.get_table_info
   ()
```

Figure 19 : Charge le schéma de la BD

Charge le schéma de la base de données pour générer des requêtes SQL adaptées.

G. Génération de Requêtes SQL avec l'IA :

Python

```
1 sql_generation_prompt = PromptTemplate(
2     input_variables=["question", "database_schema"],
3     template="""
4     Generate an SQLite valid SQL query based on the given question and database s
5     chema.
6     """
7     """
8     ### Database Schema:
9     {database_schema}
10
11     ### User Question:
12     {question}
13
14     ### Task: Return only the query based on the user question (without sql in th
15     e beginning) and database schema nothing else.
16
17     """
18     """
19     ### SQL Query:
20     {sql_query}
21 """
22 sql_generation_chain = LLMChain(llm=llm, prompt=sql_generation_prompt)
```

Figure 20 : Model de requête SQL

Crée un modèle de requête pour transformer une question en requête SQL valide.

Python

```
1 result_generation_prompt = PromptTemplate(
2     input_variables=["question", "sql_query", "sql_result"],
3     template="""
4     """
5     """
6     ### User Question:
7     {question}
8
9     ### SQL Query Used:
10    {sql_query}
11
12    ### SQL Query Result:
13    {sql_result}
14
15    **Generate a natural language response based on the SQL result.**
16
17    """
18    """
19    """
20    """
21    """
22    result_generation_chain = LLMChain(llm=llm, prompt=result_generation_prompt)
```

Figure 21 : Génération d'une réponse en NLP

Génère une réponse en langage naturel basée sur le résultat SQL.

H. Chat & Exécution de la Requête :

Python

```
1 if "messages" not in st.session_state:
2     st.session_state.messages = []
```

Figure 22 : Commande pour l'historique

Initialise un historique des messages si inexistant.

Python

```
1 user_query = st.chat_input("Pose une question sur ta base de données...")
```

Figure 23 : Commande pour écrire en NLP

Permet à l'utilisateur d'écrire une question en langage naturel.

Python

```
1 if user_query
2     y:sql_query = sql_generation_chain.invoke({
3         "question": user_query,
4         "database_schema": database_schema
5     }) "text"].strip()
6 [
7     df = pd.read_sql_query(sql_query, conn)
8     st.dataframe(df)
9
10    final_response = result_generation_chain.invoke({
11        "question": user_query,
12        "sql_query": sql_query,
13        "sql_result": df.to_string(index=False)
14    }) "text"
15 [
16    with st.chat_message("assistant"):
17        st.markdown(final_response)
```

Figure 24 : Convertit la question en SQL

Convertit la question en SQL, exécute la requête et affiche les résultats sous forme de DataFrame.

L'IA génère une réponse textuelle pour interpréter les résultats.

I. Footer & Ajout d'une Image dans la Sidebar :

Explication : Cette partie a déjà été expliquée dans le launcher.

J. Conclusion :

- Authentification sécurisée pour restreindre l'accès aux bases de données.
- Utilisation d'une IA (ChatGroq) pour transformer des questions en requêtes SQL.
- Affichage interactif des résultats SQL sous forme de tableaux.

Interface utilisateur intuitive avec sidebar et footer.

4. Explication du Code Local.py

A. Importation des Bibliothèques :

Python

```
1 import os
2 import streamlit as st
3 from premsql.generators import Text2SQLGeneratorH
4 from premsql.executors import SQLiteExecutor
5 from premsql.agents.baseline.workers import BaseLineText2SQLWorker
```

Figure 25 : Importation des Bibliothèques

Explication :

- os : Pour gérer les interactions avec le système (fichiers, variables d'environnement).
- streamlit as st : Framework pour créer l'interface utilisateur.
- Text2SQLGeneratorHF : Générateur de requêtes SQL basé sur Hugging Face.
- SQLiteExecutor : Exécute les requêtes SQL sur une base de données SQLite.
- BaseLineText2SQLWorker : Gère la conversion des questions en requêtes SQL et leur exécution.

B. Affichage de l'image en haut de la page :

Explication : Cette partie a déjà été expliquée dans les précédents codes.

C. Authentification des utilisateurs :

Explication : Cette partie a déjà été expliquée dans les précédents codes.

D. Affichage de la Barre Latérale (Sidebar) :

Explication : Cette partie a déjà été expliquée dans les précédents codes.

E. Initialisation des Générateurs Text-to-SQL :

Python

```
1 text2sql_generator = Text2SQLGeneratorHF(
2     model_or_name_or_path="premai-io/prem-1B-SQL",
3     experiment_name="text2sql_worker",
4     type="test"
5 )
```

Figure 26 : Initialisation des Générateurs Text-to-SQL

Explication :

- Text2SQLGeneratorHF utilise le modèle "premai-io/prem-1B-SQL" pour transformer une question en requête SQL.
- experiment_name="text2sql_worker" : Définit un nom d'expérimentation pour suivre les interactions.

- `type="test"` : Spécifie le mode de fonctionnement du générateur.

F. Sélection de la Base de Données :

Python

```
1 selected_db = st.sidebar.selectbox("Sélectionnez une base de données",
2 st.session_state.allowed_databases)
3 db_connection_uri = f"sqlite:///selected_db"
```

Figure 27 : Sélection de la Base de Données

Explication :

- Affiche une liste déroulante (selectbox) avec les bases de données autorisées pour l'utilisateur.
- Construit l'URI SQLite (`db_connection_uri`) pour établir une connexion avec la base sélectionnée.

G. Initialisation du Worker (Traitement Text-to-SQL) :

Python

```
1 text2sql_worker = BaseLineText2SQLWorker(
2 db_connection_uri=db_connection_uri,
3 generator=text2sql_generator,
4 executor=SQLiteExecutor()
5 )
```

Figure 28 : Traitement Text-to-SQL

Explication :

- `BaseLineText2SQLWorker` : Enchaîne le générateur Text-to-SQL avec l'exécution SQL.
- `db_connection_uri` : Connexion à la base SQLite.
- `generator=text2sql_generator` : Utilise le modèle Hugging Face pour convertir le texte en SQL.
- `executor=SQLiteExecutor()` : Exécute les requêtes SQL générées.

H. Gestion des Messages et Historique :

python

```
1 if "messages" not in st.session_state:
2     st.session_state.messages = []
3 if "results" not in st.session_state:
4     st.session_state.results = []
```

Figure 29 : Gestion des Messages et Historique

Stocke l'historique des messages (requêtes et réponses) dans `st.session_state`.
Stocke les résultats SQL dans `st.session_state.results`.

I. Affichage des Messages Précédents :

python

```
1 for message in st.session_state.messages:
2     with st.chat_message(message["role"]):
3         st.markdown(message["content"])
```

Figure 30 : Affiche les messages

Affiche les messages précédents de l'utilisateur et de l'IA sous forme de chat.

Python

```
1 for result in st.session_state.results:
2     st.subheader("Résultats de la requête :")
3     st.dataframe(result)
```

Figure 31 : Affiche les résultats SQL

Affiche les résultats SQL précédemment générés sous forme de tableau.

J. Traitement d'une Nouvelle Question :

Python

```
1 user_query = st.chat_input("Pose une question sur ta base de données...")
```

Figure 32 : Champ de saisie en NLP

Champ de saisie où l'utilisateur pose une question en langage naturel.

Python

```
1 if user_query
2     st.session_state.messages.append({"role": "user", "content": user_query})
3     with st.chat_message("user"):
4         st.markdown(user_query)
```

Figure 33 : Stocke la requête dans l'historique

Stocke la requête dans l'historique et l'affiche dans le chat.

K. Génération et Exécution de la Requête SQL :

Python

```
1 try:
2     text2sql_response = text2sql_worker.run(
3         question=user_query,
4         temperature=0.01
5     )
6     sql_query = text2sql_response.sql_string if text2sql_response else ""
```

Figure 34 : Génération et Exécution de la Requête SQL

Explication :

- text2sql_worker.run(...) : Convertit la question en requête SQL.
- temperature=0.01 : Paramètre de régularisation pour réduire la variation des requêtes générées.
- Récupère la requête SQL générée.

L. Vérification et Exécution de la Requête SQL :

Python

```
1 if not sql_query or not sql_query.lower().startswith(("select", "pragma", "with")):
2     st.sidebar.error("⚠ La requête SQL générée semble incorrecte.")
3     st.stop()
```

Figure 35 : Vérification

Vérifie que la requête générée est valide et commence bien par un mot-clé SQL (SELECT, PRAGMA, WITH).

Python

```
1 st.sidebar.write(f"📄 Requête SQL générée : {sql_query}")
```

Figure 36 : Affiche la Requête SQL

Affiche la requête SQL générée dans la barre latérale.

Python

```

1  try:
2      df_to_show = text2sql_response.show_output_dataframe()
3      if df_to_show.empty:
4          bot_response =
5          "⚠️ Aucun résultat trouvé. Vérifiez que la base contient bien des données."
6      else:
7          st.subheader("Résultats de la requête :")
8          st.dataframe(df_to_show)
9          st.session_state.results.append(df_to_show)
10         bot_response = "📊 Résultats mis à jour"
11 except Exception as sqlError:
12     bot_response = f"❌ Erreur SQL : {str(sql_error)}"

```

Figure 37 : Exécute la requête SQL

Exécute la requête SQL générée et affiche les résultats sous forme de DataFrame.
Si aucune donnée n'est retournée, affiche un message d'avertissement.
Si une erreur SQL se produit, l'affiche dans la réponse.

Python

```

1  st.session_state.messages.append({"role": "assistant", "content": bot_response})
2  with st.chat_message("assistant"):
3      st.markdown(bot_response)

```

Figure 38 : Ajoute la réponse du bot

Ajoute la réponse du bot (résultat SQL ou erreur) à l'historique et l'affiche dans le chat.

M. Footer Fixe et Image dans la Sidebar :

Explication : Cette partie a déjà été expliquée dans les précédents codes.

N. Conclusion :

- Authentification sécurisée avec des rôles et des permissions spécifiques aux bases de données.
- Utilisation d'un modèle Text-to-SQL (PremSQL) pour convertir du texte en requêtes SQL.
- Exécution et affichage des requêtes SQL directement dans Streamlit.
- Gestion de l'historique des requêtes et affichage interactif des résultats.
- Interface intuitive avec sidebar et footer.

5. Présentation de l'Interface Streamlit des Applications ChatINDUS :

L'interface des applications ChatINDUS repose sur Streamlit, un framework Python permettant de créer des applications interactives et intuitives. Elle est conçue pour offrir une expérience utilisateur fluide, avec une organisation claire des fonctionnalités.

A. Structure Générale de l'Interface :

L'interface est divisée en trois parties principales :

a) En-tête et Image Fixe

- Affichage d'une bannière ou d'un logo en haut de la page pour identifier l'application.
- Si l'image est manquante, un message d'avertissement est affiché.

B. Barre Latérale (Sidebar) :

- Titre et Branding : Indique le nom de l'application et son mode d'exécution (Local ou Cloud).
- Outils et Paramètres :
 - Accès à un visualiseur de bases de données externe.
 - Zone d'information sur la génération de requêtes SQL à partir du langage naturel. Sélection d'une base de données parmi celles autorisées.

C. Zone Centrale (Corps de l'Application) :

- Authentification
 - Un système de connexion avec différents rôles et permissions.
 - Les utilisateurs peuvent accéder uniquement aux bases de données autorisées.
- Saisie de Requêtes en Langage Naturel
 - L'utilisateur peut poser une question sur une base de données via un champ de texte interactif.
 - L'application convertit la requête en SQL valide et exécute cette dernière sur la base sélectionnée.
- Affichage des Résultats
 - Présentation des données sous forme de tableau interactif.
 - Conservation de l'historique des requêtes et des réponses sous forme de chat.
- Messages et Feedback
 - Affichage des messages précédents pour suivre l'historique de la conversation.
 - En cas d'erreur SQL ou de problème de connexion, des messages d'alerte sont affichés.

D. Expérience Utilisateur et Ergonomie :

- Navigation intuitive grâce à la séparation des zones fonctionnelles (authentification, requêtes, affichage des résultats).
- Réponses dynamiques qui mettent à jour les résultats en temps réel.

- Retour visuel immédiat avec des notifications en cas d'erreur ou de succès.
- Accessibilité optimisée avec une gestion des rôles et des permissions adaptées aux utilisateurs.

E. Élément Visuel Important : Le Footer :

- Un pied de page fixe affichant les auteurs du projet et le superviseur.
- Permet de donner une touche professionnelle et informative à l'application.

F. Conclusion :

L'interface ChatINDUS propose une approche modulaire et intuitive pour interagir avec des bases de données en langage naturel. Grâce à Streamlit, l'interface assure une navigation fluide et une exécution rapide des requêtes SQL, le tout avec une expérience utilisateur optimisée.

6. Axes d'Amélioration pour les Applications ChatINDUS :

Après analyse des trois codes Streamlit utilisés pour ChatINDUS (Launcher, Cloud & Local), voici plusieurs axes d'amélioration qui pourraient optimiser l'expérience utilisateur, améliorer les performances et renforcer la sécurité.

A. Amélioration de l'Expérience Utilisateur :

1) Optimisation de l'Interface Graphique :

- Ajouter une barre de progression lors du traitement des requêtes SQL pour améliorer la perception du temps d'exécution.
- Utiliser des animations ou effets visuels pour rendre l'application plus interactive.
- Implémenter une thématique personnalisable permettant à l'utilisateur de choisir entre un mode clair/sombre.

2) Meilleure Gestion des Messages :

- Ajouter une fonctionnalité de filtrage pour retrouver rapidement une requête spécifique dans l'historique.
- Offrir la possibilité de supprimer des messages spécifiques dans l'historique de chat.
- Permettre l'exportation des conversations en fichiers texte ou CSV pour une réutilisation ultérieure.

3) Feedback Utilisateur Plus Dynamique :

- Afficher un résumé simplifié des résultats SQL avant le tableau détaillé.
- Ajouter un message audio ou une notification visuelle en cas d'échec de la requête SQL.
- Intégrer une zone d'aide contextuelle expliquant comment poser des questions efficacement en langage naturel.

B. Optimisation des Performances :

1) Amélioration de la Rapidité des Requêtes SQL :

- Implémenter une mise en cache des requêtes pour éviter d'exécuter plusieurs fois la même requête.
- Ajouter une vérification syntaxique SQL avant l'envoi pour éviter des erreurs inutiles et améliorer la précision.
- Utiliser des index SQL pour accélérer les requêtes sur des bases de données volumineuses.

2) Optimisation des Modèles de Génération SQL :

- Comparer ChatGroq vs PremSQL pour identifier le modèle le plus performant en termes de précision et de rapidité.
- Tester différentes valeurs de température pour réduire la génération de requêtes SQL incorrectes.
- Ajouter une fonctionnalité d'apprentissage permettant d'améliorer progressivement la pertinence des requêtes générées.

3) Gestion des Erreurs et des Anomalies :

- Implémenter une reprise sur erreur automatique, permettant de proposer une alternative lorsque la requête SQL échoue.
- Ajouter une analyse des logs d'exécution pour identifier les sources de ralentissement ou d'anomalies dans les requêtes.
- Utiliser des tests unitaires pour valider la robustesse des transformations Text-to-SQL.

C. Sécurité et Gestion des Accès :

1) Renforcement de l'Authentification :

- Remplacer l'authentification basique par un système sécurisé avec chiffrement des mots de passe.
- Ajouter une double authentification (2FA) pour les comptes ayant un accès critique aux bases de données.
- Implémenter une gestion avancée des rôles avec plus de granularité (ex : lecture seule, modification, suppression).

2) Protection contre les Requêtes Malicieuses :

- Ajouter un filtre anti-injection SQL pour empêcher toute tentative d'attaque sur la base de données.
- Restreindre certaines requêtes SQL sensibles (ex : `DROP TABLE`, `DELETE FROM`, etc.).
- Mettre en place un système de logs et alertes pour détecter les activités suspectes.

3) Sécurisation des Clés API et des Données Sensibles :

- Stocker la clé API Groq de manière sécurisée via des variables d'environnement encryptées.
- Vérifier que les bases de données sélectionnées par les utilisateurs sont bien sécurisées et n'exposent pas d'informations sensibles.
- Restreindre l'accès aux fichiers `.sqlite` via une gestion avancée des permissions.

D. Déploiement et Accessibilité :

1) Amélioration du Déploiement Cloud :

- Tester plusieurs solutions de déploiement (Streamlit Cloud, AWS, Heroku, Docker) pour identifier l'option la plus stable.
- Implémenter un système de mise à jour automatique, permettant aux utilisateurs d'accéder à la dernière version sans redéploiement manuel.
- Ajouter un mode hors ligne, où certaines fonctionnalités peuvent fonctionner même sans connexion internet.

2) Optimisation des Logs et de la Maintenance :

- Mettre en place un système de monitoring pour surveiller les performances et la stabilité de l'application.
- Permettre aux administrateurs d'exporter des rapports d'activité pour analyser l'utilisation de la plateforme.
- Intégrer une page de diagnostic où les utilisateurs peuvent voir l'état du serveur, des bases de données et des modèles IA.

3) Accessibilité et Compatibilité :

- Vérifier que l'application est compatible avec tous les navigateurs modernes.
- Ajouter une prise en charge des écrans mobiles et tablettes pour une utilisation plus flexible.
- Permettre l'intégration avec d'autres plateformes via API (ex : Google Sheets, Power BI).

7. Conclusion

Ces améliorations permettront de rendre les applications ChatINDUS plus performantes, sécurisées et intuitives. Elles garantiront une expérience fluide aux utilisateurs, tout en optimisant la gestion des requêtes SQL et la sécurité des bases de données.

III. Conclusions et perspectives :

Ce projet nous a offert une expérience enrichissante, nous permettant de renforcer nos compétences en programmation, en gestion de projet et en travail collaboratif, tout en explorant des solutions innovantes pour optimiser l'accès aux bases de données. Nous avons également découvert et appris à maîtriser de nouvelles technologies et concepts pour la première fois, ce qui nous a permis d'élargir nos connaissances et de mieux comprendre les défis liés à la conception et à l'optimisation d'une solution informatique. ChatINDUS a été développé sous deux formes : une solution locale, garantissant plus de confidentialité et d'autonomie, et une solution cloud, offrant une plus grande accessibilité et évolutivité. Chaque approche présente ses avantages et ses contraintes, ouvrant la voie à de futures améliorations en termes de performance, de sécurité et d'expérience utilisateur.

Nous souhaitons exprimer notre profonde gratitude à notre encadrant, Estéban Ruiz Bautista, pour son accompagnement et ses conseils précieux tout au long du projet. Nous remercions également le personnel du laboratoire LISIC, ainsi que l'ensemble des enseignants et du personnel de l'EILCO, dont l'expertise et le soutien nous ont permis d'acquérir de nouvelles compétences et de relever des défis techniques et méthodologiques. Ce projet nous a fortement motivés à poursuivre le développement de ChatINDUS, en explorant des pistes d'amélioration et d'optimisation afin d'en faire un outil toujours plus performant et adapté aux besoins des entreprises. Nous sommes convaincus que ce travail constitue une base solide pour de futures évolutions, et nous restons enthousiastes à l'idée d'approfondir nos recherches et d'enrichir davantage cette solution.

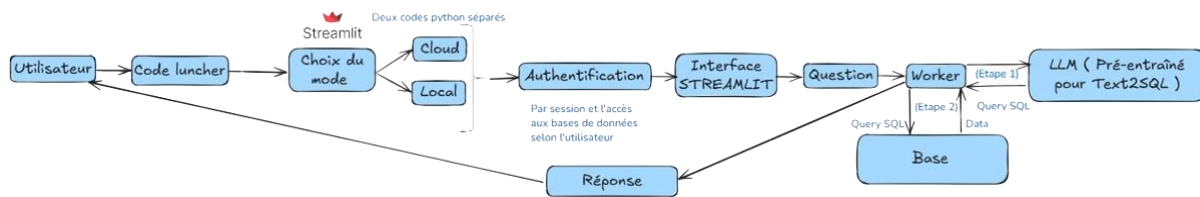


Figure 39 : Schéma générale du projet.

IV. Annexes

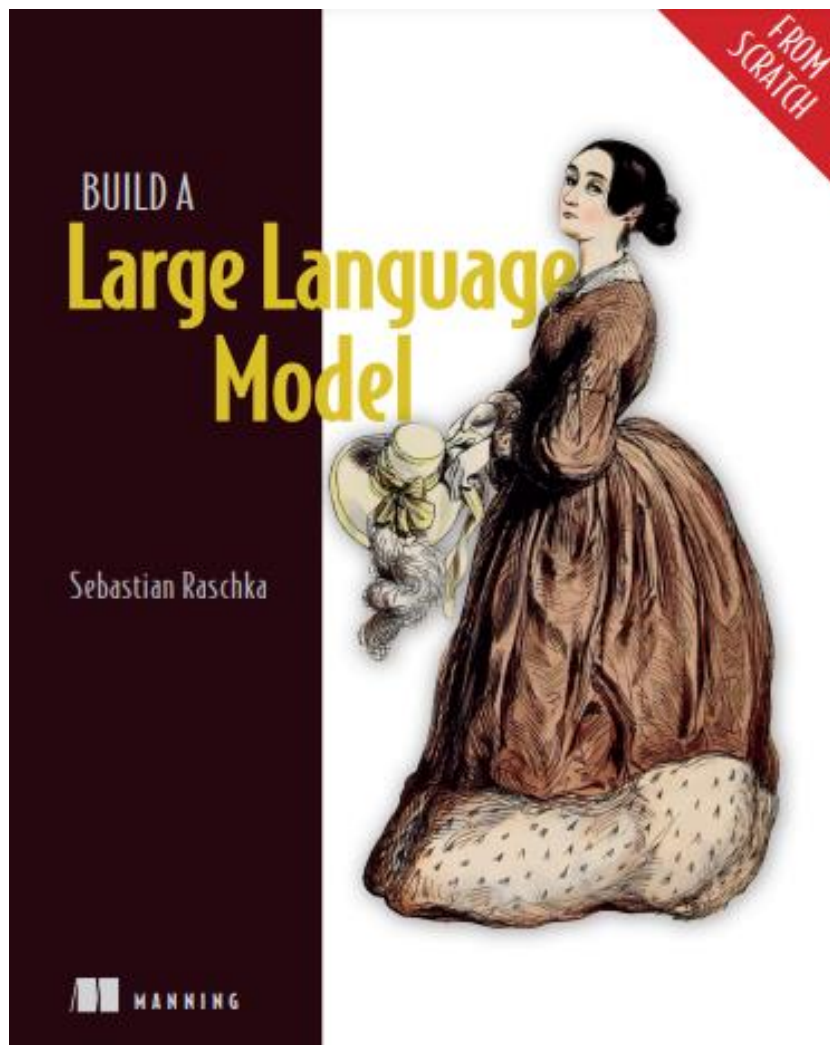


Figure 40 : Annexe 1